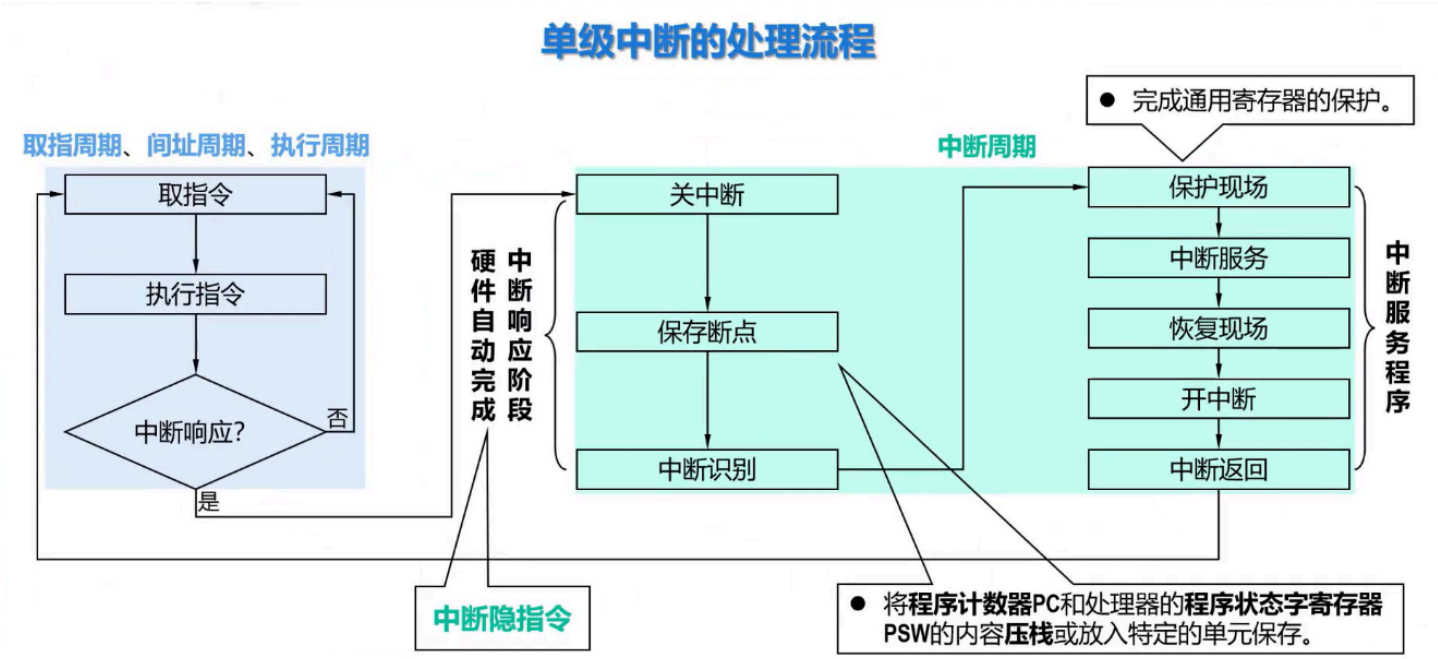
8.10 单级中断处理流程

考纲定位：计算机组成原理 · 第 8 章 I/O 系统（8.10）｜重要度 ★★★★｜常见考法：选择题（中断隐指令三件事、ISR 五步顺序、断点 vs 现场），高频考点

一句话核心：单级中断 = 指令边界响应 → 硬件三连（关中断、保存断点 PC+PSW、中断识别）→ 软件五连（保护现场、中断服务、恢复现场、开中断、中断返回），服务期间全程关中断

一、单级中断处理流程图



1.1 左侧：正常执行循环与响应时机

CPU 平时在“取指周期 →（间址周期）→ 执行周期”的流水里循环。每条指令执行完毕，硬件检查菱形框“中断响应？”：

- 否 → 回到取指，继续下一条
- 是 → 进入中断周期（绿色区）

两个必考细节：

1. 响应边界是“指令执行周期结束”，CPU 绝不会在指令执行到一半时被可屏蔽中断打断——中断请求最多等到这条指令执行完
2. 响应需要同时满足：有中断请求 + CPU 开中断（IF=1）+ 请求未被屏蔽，缺一不可

这里还要建立一个重要概念：中断周期是 CPU 的第四种工作周期，与取指、间址、执行周期并列。也就是说，“响应中断”在 CPU 眼里也是一段时间内的工作阶段，并非凭空插入

1.2 中间：中断周期 = 中断隐指令

进入中断周期后，CPU 硬件自动完成三个动作，图下方标注“中断隐指令”——它不是一条真实的指令，没有操作码、不出现在程序里、程序员不可见，纯粹是硬件在响应中断时的一套固定微操作序列

- **动作① 关中断**：清 IF 位。为什么放第一步？为了保证后续“保存断点”的原子性——如果断点压栈压到一半又被新中断打断，栈里的断点就废了，原程序再也回不去了。关中断就是给接下来的硬件操作上一把锁
- **动作② 保存断点**：按图中注释，把 PC 和 PSW 一起压栈（或存入特定单元）。PC 是“回来后从哪继续取指”，PSW 是“回来后恢复什么运行状态（含 IF 位本身）”。注意这是硬件自动压栈，不是软件做的
- **动作③ 中断识别**：判明是哪个中断源请求，并找到对应中断服务程序的入口地址——这一步对应 8259A 流程图里的第二个 INTA 脉冲：8259A 把中断类型号放上数据总线，CPU 取回后查中断向量表，得到 ISR 入口地址送入 PC。到此硬件任务全部完成，控制权正式移交软件

隐指令只做这三件事，多一件都没有——保护现场不属于中断隐指令，这是 408 最爱挖的分界线

1.3 右侧：中断服务程序五步

从“保护现场”开始，全是程序员编写的 ISR 在跑：

1. **保护现场**：把通用寄存器内容压栈（图中气泡注明“完成通用寄存器的保护”）。注意与断点的分工：断点（PC + PSW）硬件已存，现场（通用寄存器）软件来存——因为 ISR 要用这些寄存器，不保护就会破坏被中断程序的运行环境
2. **中断服务**：干正事，比如读走键盘缓冲、启动打印机。单级中断下这一段全程关着中断，任何新请求都只能等
3. **恢复现场**：把通用寄存器从栈里弹回。注意它必须发生在开中断之前——如果先开中断再恢复现场，恢复到一半被新中断打断，寄存器又被别人改了，现场就乱了
4. **开中断**：置回 IF 位。这是单级中断的标志性位置——开中断只出现在服务程序末尾。为什么必须有这一步？因为隐指令阶段关了中断，如果一直不开，返回主程序后 CPU 将永远不再响应任何中断，系统就“聋”了
5. **中断返回**：执行 IRET，硬件自动弹出断点（PC 和 PSW），回到被中断的程序，图上的箭头绕回左侧“取指”，回到主循环

1.4 时间线串讲

CPU 正在执行主程序，每执行完一条指令就瞄一眼中断请求线。某次执行完，发现有请求且 IF=1，于是进入中断周期：硬件先关中断锁住现场，再把 PC 和 PSW 压栈保住退路，然后通过中断识别找到服务程序入口；接着软件登场，先压栈保护通用寄存器，再专心处理中断请求（全程无打扰），处理完弹栈恢复寄存器，然后开中断恢复响应能力，最后 IRET 弹回 PC 和 PSW，从断点处继续执行主程序

1.5 单级中断 vs 多重中断

对比项	单级中断（本图）	多重中断
服务程序执行期间	全程关中断，不允许被打断	保护现场后立即开中断，允许更高优先级请求插入
开中断的位置	恢复现场之后、中断返回之前（末尾一次）	保护现场之后开一次（放行嵌套）；恢复现场前关一次；恢复完再开，然后返回（出现两次）
中断嵌套	不可能发生	可形成嵌套链
硬件代价	小	需要优先级判别逻辑（如 8259A 的 PR）

记忆口诀：单级“一关到底，最后才开”；多级“早开晚关，夹住现场”——多级中“恢复现场前必须先关中断”，原理与单级中“恢复现场前不能开中断”完全一致

1.6 易错点清单

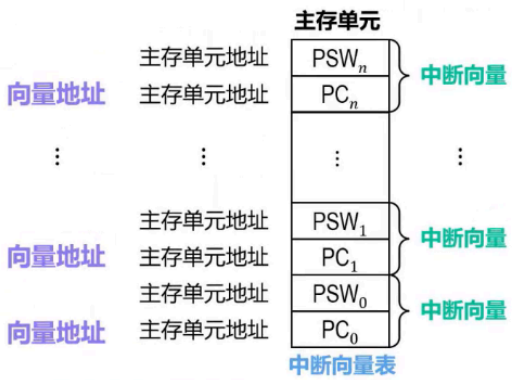
1. **中断隐指令 vs 中断服务程序的分界**：隐指令 = 关中断、保存断点、中断识别（硬件）；保护现场及之后 = 软件。2010 年真题就考“哪些操作在 ISR 内完成”，答案里绝不能混入隐指令的动作
2. **断点 vs 现场**：断点 = PC + PSW（硬件压栈），现场 = 通用寄存器（软件压栈），两者保存的时机和执行者都不同
3. **保护现场不在隐指令里**：同理，恢复断点也不由软件做——IRET 执行时硬件自动弹 PC 和 PSW，ISR 里不用手写
4. **关中断必须最先做**：它保护的是“保存断点”这个动作本身
5. **开中断的位置是单级/多级判断题的判据**：看到流程图中“开中断”出现在保护现场之后 → 多级；出现在恢复现场之后 → 单级
6. **中断响应不发生在指令中间**：选项说“CPU 随时可响应中断”必错（NMI 也只是在指令边界优先级更高地被响应，不是插在指令中间）
7. **中断识别 ≠ 软件查询才叫识别**：向量中断由硬件排队器 + 编码器直接给出向量地址（对应 8259A 送类型号），软件查询法才是服务程序里逐个查状态位

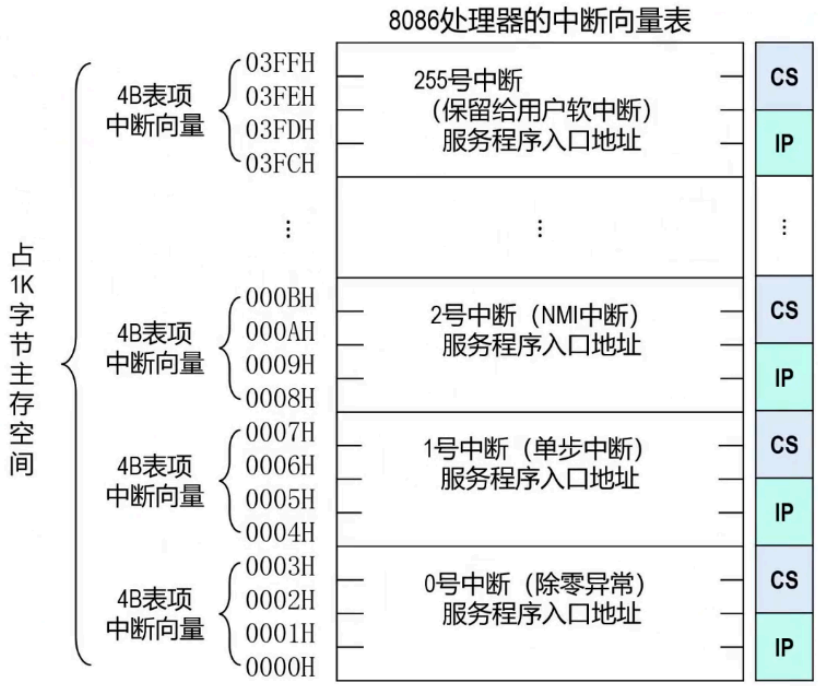
一句话收束：指令边界响应 → 硬件三连（关中断/存断点/找入口）→ 软件五连（护现场/办正事/复现场/开中断/回家）；单级中断的全部精髓，就是中间那段“办正事”时大门紧锁，直到收拾完行李才重新开门

二、中断向量、中断向量表、向量地址

中断向量、中断向量表、向量地址

- CPU将**中断号**作为**索引**，在**中断向量表**中查找**中断服务程序的入口地址**，以便实现程序的转移。
- **中断向量**：一般将**中断服务程序的入口地址PC**和**程序状态字PSW**合称为中断向量。
  - 某些计算机的中断服务程序需要载入**PSW**才能运行。
  - 对于**不需要载入PSW**即可运行中断服务程序的计算机，中断向量就是**中断服务程序的入口地址PC**。
- **中断向量表**：由**若干中断向量**构成（中断向量的集合）。
  - 通常**存放在主存中**，在操作系统引导过程中初始化。
  - 利用**中断号**对其进行索引访问。
- **向量地址**：中断向量表中各表项的地址，也称为**中断指针**。





- 将**中断号n作为索引**，对中断向量表中的中断向量（4B表项）进行访问。
  - **中断向量在中断向量表中的位置**：$n \times 4$
- 中断向量地址的高2个字节的内容，用于更新到**代码段寄存器CS**。
- 中断向量地址的低2个字节的内容，用于更新到**指令指针寄存器IP**。
- 8086处理器中，**代码段寄存器CS**和**指令指针寄存器IP**共同决定了CPU将要执行的下一条指令在**主存中的物理地址**。
  - **代码段寄存器CS**：存放当前代码段的起始地址，CPU在计算物理地址时，会将其左移4位。
  - **指令指针寄存器IP**：存放相对于当前代码起始地址的偏移地址。
  - **主存中的物理地址**：(CS) << 4 + (IP)

408 口径

术语	含义	易错点
中断类型号	中断控制器（如 8259A）送给 CPU 的编号，用来索引中断向量表	类型号不是入口地址
中断向量	中断服务程序的入口地址本身	不要把类型号或向量地址当成中断向量
中断向量表	存放所有中断向量的表，按中断类型号索引	表项内容才是入口地址
向量地址	中断向量表中存放某入口地址的单元地址	CPU 先拿到向量地址/类型号，查表后才得到入口地址

- 过程：中断类型号 → 查中断向量表 → 得到中断向量（ISR 入口地址）→ 送入 PC
- 8086 的例子：每个中断类型对应 4B 表项，高 2B 存放段基址 CS，低 2B 存放偏移量 IP；物理地址 = CS × 16 + IP
- 中断向量表通常存放在内存低地址区，由操作系统在系统引导时初始化

口径提示：本课件图注把“ISR 入口地址 PC + 程序状态字 PSW”合称中断向量；408 王道口径的中断向量通常指 ISR 入口地址本身。做题时按 408 口径：中断向量 = 入口地址，向量地址 = 向量表中存放入口地址的单元地址；“保存断点 = PC + PSW”是另一件事，二者不矛盾

三、易错辨析

易混点	核心区分
中断隐指令 vs ISR	隐指令：关中断、保存断点、中断识别（硬件）；ISR：保护现场、中断服务、恢复现场、开中断、中断返回（软件）

易混点	核心区分
断点 vs 现场	断点 = PC + PSW，硬件保存；现场 = 通用寄存器，软件保存
中断向量 vs 向量地址	中断向量是入口地址；向量地址是向量表中存放入口地址的单元地址
中断类型号 vs 中断向量	类型号是索引；中断向量是表项内容（入口地址）
单级 vs 多重	单级全程关中断，末尾才开；多重保护现场后开中断，允许嵌套
IRET vs 普通返回	IRET 弹回 PC + PSW；普通 RET 只弹返回地址
中断响应时机	指令执行完毕的边界；不在指令中途

四、考法透视

选择题常见设错点

1. 把“保护现场”算进中断隐指令
2. 认为断点只保存 PC，不保存 PSW（408 口径下断点 = PC + PSW）
3. 认为 IRET 由软件手动弹栈
4. 认为开中断在保护现场之前（单级中断）
5. 认为中断可以在指令执行中途响应
6. 把中断向量、向量地址、中断类型号混为一谈
7. 认为单级中断不能有多个中断源

流程题

- 给单级中断流程图，补全硬件三连与软件五步的顺序
- 给多重中断流程图，判断开中断/关中断/EOI 的位置
- 给中断向量表，计算表项地址或入口地址

五、知识关联

- 前置：8.7 中断的概念、8.8 中断优先级和中断屏蔽、8.9 中断请求
- 后续：8.11 多重中断的处理流程（开关中断与 EOI）
- OS 视角：中断隐指令、系统调用陷入、进程状态切换
- 全章导航：8.0 导航与术语

六、费曼自测

1. 中断隐指令完成哪三件事 → 关中断、保存断点（PC + PSW）、中断识别（引出 ISR 入口）
2. 断点和现场分别由谁保存 → 断点 PC + PSW 由硬件在中断隐指令阶段保存；现场通用寄存器由软件在 ISR 中保存

3. 单级中断的 ISR 五步顺序是什么 → 保护现场 → 中断服务 → 恢复现场 → 开中断 → 中断返回
4. 为什么单级中断要在恢复现场之后才开中断 → 防止恢复现场到一半被新中断打断，导致寄存器现场被破坏
5. 中断向量、向量地址、中断类型号有什么区别 → 类型号是索引；向量地址是向量表中存放入口地址的单元地址；中断向量是入口地址本身